

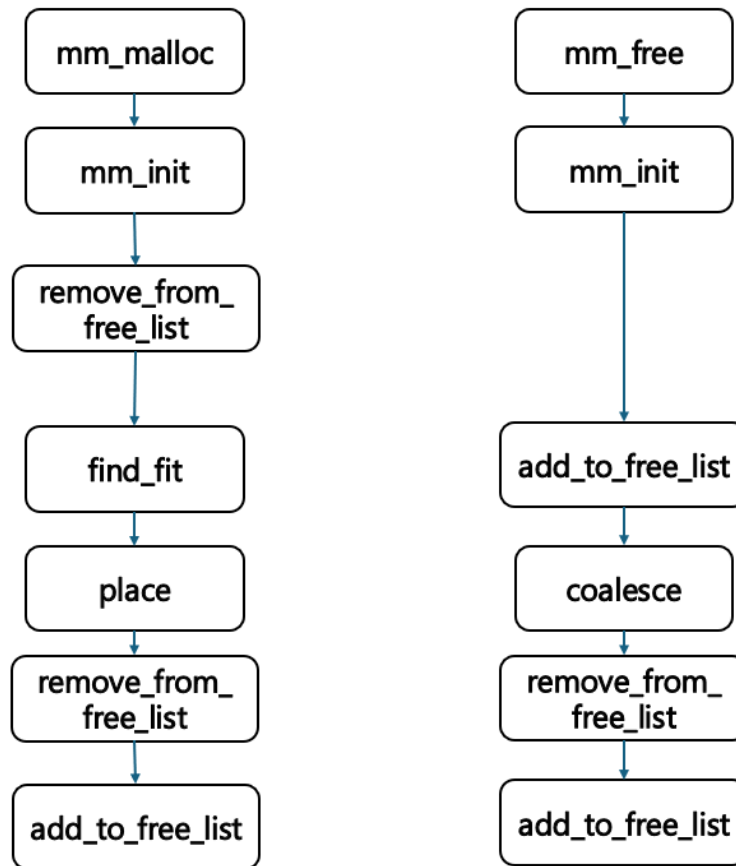
System Programming Project 4

담당 교수 : 이영민

이름 : 이동협

학번 : 20211562

1.1 Allocator 설계



2.1 추가한 전역 변수

- `static char *heap_listp`: 힙의 시작 지점(프로로그 블록의 패딩)을 가리키는 포인터
- `static char *free_listp`: 명시적 가용 리스트의 헤드 포인터, LIFO(Last-In, First-Out) 방식으로 관리

2.2 추가한 매크로 정의

- `WSIZE (4)`, `DSIZE (8)`: word 및 double word 크기를 정의
- `CHUNKSIZE (1<<12)`: heap 확장을 위한 기본 청크 크기 (4KB)
- `MAX(x, y)`: 두 값 중 큰 값을 반환
- `PACK(size, alloc)`: 크기와 할당 비트를 결합하여 header/footer 값을 생성
- `GET(p)`, `PUT(p, val)`: 특정 주소에서 word를 읽고 씸

- GET_SIZE(p), GET_ALLOC(p): header/footer에서 크기와 할당 비트를 추출
- HDRP(bp), FTRP(bp): payload pointer(bp)로부터 header 및 footer의 주소를 계산
- NEXT_BLKp(bp), PREV_BLKp(bp): 현재 블록의 bp로부터 다음/이전 블록의 bp를 계산
- PRED_PTR(bp), SUCC_PTR(bp): 가용 블록 bp로부터 PRED/SUCC 포인터가 저장될 메모리 위치를 계산
- GET_PRED(bp), GET_SUCC(bp): 가용 블록 bp에서 PRED/SUCC 포인터 값을 읽음
- SET_PRED(bp, val), SET_SUCC(bp, val): 가용 블록 bp에 PRED/SUCC 포인터 값을 설정
- ALIGN(size): 요청된 크기를 8바이트 정렬에 맞춤
- OVERHEAD (DSIZE): header와 footer의 총 크기 (8바이트)
- MIN_ALLOC_SIZE ($2 * DSIZE + DSIZE$): 최소 할당 가능한 블록 크기 (header + footer + 최소 payload(8) + PRED/SUCC 포인터 공간을 고려한 최소 크기)

2.3 주요 서브루틴 설명

- int mm_init(void):

Mallocator를 초기화하고 heap의 시작 부분에 padding, 프롤로그 header/footer, 에필로그 header를 생성하고 free_listp를 NULL로 초기화한다.

extend_heap 함수를 호출하여 초기 heap 영역을 확장하고 첫 free 블록을 생성한다.

성공 시 0, 실패 시 -1 return

- void *mm_malloc(size_t size):

size byte 이상의 payload를 갖는 블록을 할당한다. 이후, 요청 크기를 오버헤드

와 정렬을 고려하여 asize로 조정한다. find_fit 함수를 사용하여 free 리스트에서 asize를 수용할 수 있는 최적의 블록(Best-Fit)을 찾는다. 블록을 찾으면 place 함수를 호출하여 할당하고 필요시 분할한다. 이때, 적합한 블록을 찾지 못하면 extend_heap을 호출하여 heap을 확장한 후 할당을 시도한다.

- void mm_free(void *ptr):

ptr이 가리키는 블록을 해제하고 블록의 header와 footer를 frees 상태로 표시한다. 이후 coalesce 함수를 호출하여 인접한 free 블록과 병합을 시도한다.

- void *mm_realloc(void *ptr, size_t size):

ptr이 가리키는 기존 블록의 크기를 size 바이트로 변경한다. 이때, ptr이 NULL이면 mm_malloc(size)와 동일하게 동작하고 size가 0이면 mm_free(ptr)와 동일하게 동작하고 NULL을 반환한다.

1. 새로운 크기(new_asize)가 기존 블록 크기(oldsizes)보다 작거나 같을 경우:

남은 공간이 MIN_ALLOC_SIZE 이상이면, 기존 블록을 new_asize로 줄이고 남은 공간을 새로운 free 블록으로 분할하여 add_to_free_list에 추가한다. 그렇지 않으면 기존 블록을 그대로 사용하고 ptr을 반환한다.

2. 새로운 크기가 기존 블록보다 클 경우:

다음 블록이 free 상태이고, 현재 블록과 다음 블록을 병합하여 요청 크기를 만족할 수 있다면, 다음 블록을 free 리스트에서 제거하고 현재 블록을 확장하여 ptr을 반환한다. 그렇지 않다면 mm_malloc으로 새 블록을 할당하고, 기존 데이터를 복사한 후, 기존 블록을 mm_free한다.

- static void *extend_heap(size_t words):

heap을 word만큼 확장하고 확장된 영역을 free 블록으로 만든다. 이후 새로운 free 블록의 header/footer를 설정하고, 에필로그 header를 이동시킨다. 그리고 새로 생성된 free 블록에 대해 coalesce를 호출하여 이전 블록과 병합을 시도한다.

- static void *coalesce(void *bp):

bp가 가리키는 블록과 인접한 free 블록들을 병합한다. 이전 블록과 다음 블록

의 할당 상태에 따라 네 가지 케이스로 나뉜다. 병합 시, free 리스트에서 제거 및 추가 작업을 수행하여 리스트의 일관성을 유지한다.

- static void *find_fit(size_t asize):

explicit free list를 순회하며 asize를 수용할 수 있는 가장 적합한(Best-Fit) 가용 블록을 검색하고 포인터를 반환한다.

- static void place(void *bp, size_t asize):

bp가 가리키는 가용 블록에 asize 크기의 할당된 블록을 배치한다. 그리고 bp를 free 리스트에서 제거한다. 블록을 배치한 후 남은 공간이 MIN_ALLOC_SIZE 이상이면 해당 공간을 새로운 free 블록으로 분할하고 coalesce를 호출하여 병합한다.

남은 공간이 MIN_ALLOC_SIZE보다 작으면 남은 공간은 내부 fragmentation으로 처리하고 전체 블록을 할당한다. 블록 배치 시 memset(bp, 0, MIN_ALLOC_SIZE);를 호출하여 할당된 블록의 첫 MIN_ALLOC_SIZE 바이트를 0으로 초기화하여 이전 free 블록의 PRED/SUCC 포인터 잔여물이나 mdriver 패턴을 제거한다.

- static void add_to_free_list(void *bp):

새로운 가용 블록 bp를 explicit free list head에 추가한다. 이전 헤드의 PRED 포인터와 새 블록의 SUCC 포인터를 적절히 업데이트한다.

- static void remove_from_free_list(void *bp):

free list에서 bp가 가리키는 블록을 제거한다. 그리고 블록의 위치(헤드, 중간, 끝)에 따라 PRED/SUCC 포인터 연결을 적절히 조정한다.